API Rate Limiter and Quota Service

Technical Overview

FastAPI · Redis · PostgreSQL · Nginx · Docker

In one paragraph. This service enforces per-API-key request quotas — the mechanism behind paid API tiers. Free customers get 10 requests a minute, Pro 100, Enterprise 1000, the way Stripe or OpenAI limit their APIs. The quota check runs as FastAPI middleware, so a rejected request never reaches a route handler. Counters live in Redis; plan tiers live in PostgreSQL. The interesting engineering problem is a race condition, covered in Section 5.

1. Why rate limiters exist

- **Survival.** One client writes a buggy loop and sends 10,000 requests a second. The server saturates and *everyone's* requests fail, not just theirs.
- **Revenue.** If you sell an API, **the limit is the product.** No limiter means no tiers and no way to charge. The rate limiter is effectively the cash register.
- **Abuse.** Someone trying 5,000 passwords against a login endpoint.

Why not just use nginx's built-in limiter?

Nginx ships `limit_req`, and this project deliberately does not use it. `limit_req` keys on the client's IP address. It has no concept of an API key, no idea that one customer is on Pro and another on Free, and no way to find out, because that fact is a Postgres row that changes when someone upgrades. Changing a limit would mean editing `nginx.conf` and reloading.

So nginx stays a plain reverse proxy, and the quota decision happens one hop later where the plan data is reachable.

2. Architecture

```
client
  |
  v
nginx  (port 8080)          <- the ONLY published port
  |
  v
FastAPI  (4 worker processes)
  |
  |  MIDDLEWARE, runs before routing:
  |    1. read the X-API-Key header
  |    2. find the plan   -> Redis cache, Postgres on a miss
  |    3. check the quota -> Redis, atomic Lua script
  |
  +-- over quota?   -> 429 + Retry-After     [STOPS HERE]
  +-- under quota?  -> the route handler runs
       |
       v
  Redis (counters)      PostgreSQL (api keys + plans)
```

Four containers

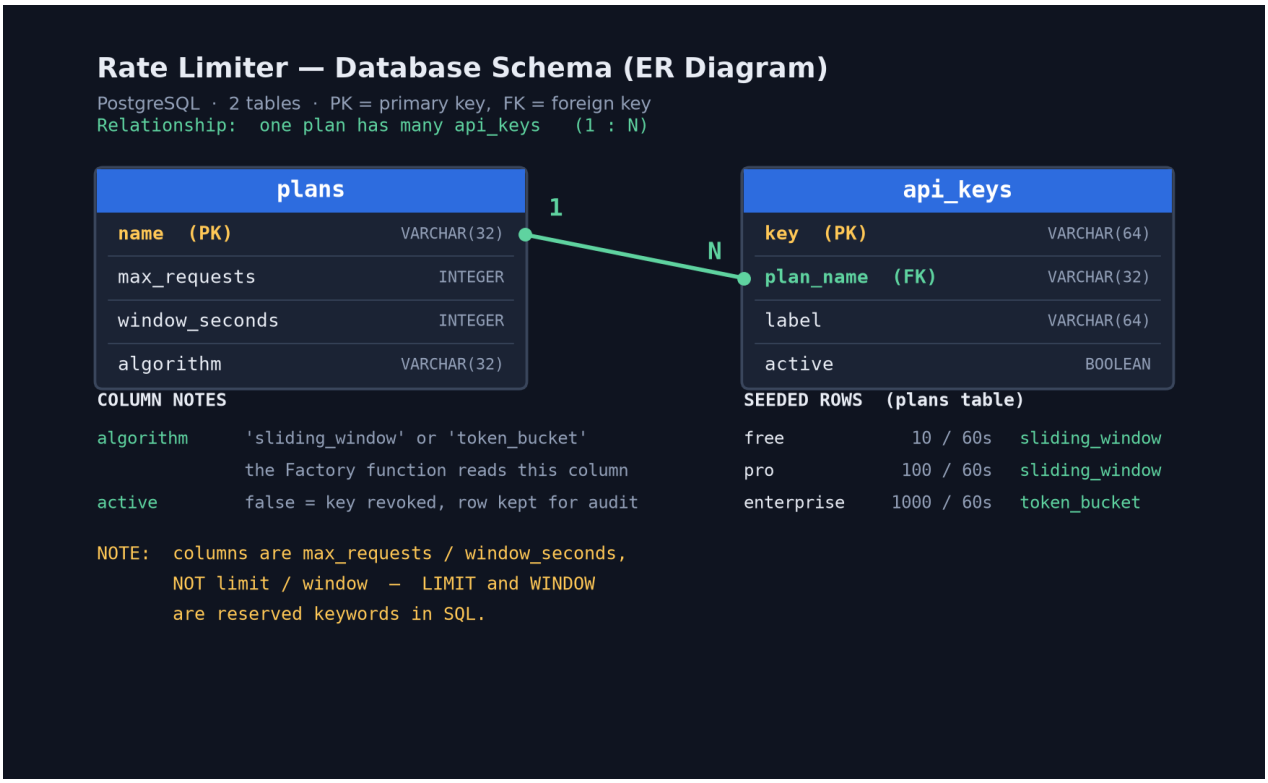
Container	Job	Reachable from outside?
`gateway`	nginx reverse proxy	**Yes — port 8080**
`api`	FastAPI, 4 uvicorn workers	No
`redis`	request counters, plan cache	No
`postgres`	API keys and plan tiers	No

Only the gateway publishes a port, so the database is not exposed at all.

Why the quota check is middleware

Middleware runs *before* routing. A rejected request therefore never opens a database session and never runs any business logic — it costs one Redis round trip and nothing else. Putting the check on individual routes instead would mean remembering to add it to every new endpoint, and the first one forgotten is an unprotected endpoint.

3. Database schema



Two tables, one-to-many: a plan has many API keys.

The key design point is that **a customer's limit is a database row, not configuration.** Upgrading someone from Free to Pro is a single `UPDATE` that takes effect immediately — no config edit, no redeploy, no restart.

Note: the columns are max_requests and window_seconds, not limit and window — LIMIT and WINDOW are reserved keywords in SQL.

Seeded plans

Plan	Limit	Algorithm	Why
free	10 / 60s	sliding window	hard ceiling
pro	100 / 60s	sliding window	hard ceiling
enterprise	1000 / 60s	token bucket	tolerates legitimate bursts

4. The two algorithms

There are five standard rate limiting algorithms. This project implements two, chosen because they make opposite trade-offs.

Sliding window log — used for Free and Pro

A Redis sorted set holds one timestamp per request. Each check drops entries older than the window and counts what remains, so the window genuinely slides.

```
1. ZREMRANGEBYSCORE  drop timestamps older than the window
2. ZCARD             count what is left
3. over the limit?   read the OLDEST entry to compute Retry-After
4. under the limit?  ZADD the current timestamp
5. PEXPIRE           so an idle key deletes itself
```

Why not a fixed window? A fixed window keeps one counter per clock minute. With a limit of 100/min, a client can send 100 requests at 11:59:59 and 100 more at 12:00:01 — **200 requests in two seconds**, and both minutes are technically legal. That is the *boundary burst* problem. A sliding window counts the trailing 60 seconds continuously, so the burst is rejected.

Token bucket — used for Enterprise

A bucket holds 1000 tokens and refills continuously at 1000/60,000 tokens per millisecond — about **16.7 tokens per second**. Each request spends one. Tokens available means allowed; empty means denied.

Think of it as prepaid phone credit that tops up automatically: stay quiet and your balance fills, then you can spend it in one burst.

Why Enterprise gets this: a business running a nightly batch job legitimately fires 500 requests at once. That is expected behaviour, not abuse. A sliding window would reject it; a token bucket that has been idle is full and lets it through, then throttles to the refill rate.

> **Sliding window = a hard ceiling. Token bucket = bursts allowed, the long-run average bounded.**

5. The core problem: a race condition

Why the obvious implementation is broken

The natural way to write a rate limiter is: read the count, compare it to the limit, increment. Under concurrency that is wrong:

```
Request A: reads count = 9    ->  9 < 10, allow
Request B: reads count = 9    ->  9 < 10, allow     <-- both got in
Request A: writes 10
Request B: writes 10                                <-- 11 allowed
```

Both read *before* either wrote. This is a **check-then-act race condition**.

Why a normal lock does not fix it

The service runs **four separate worker processes** with no shared memory, and in production it would be N replicas across N machines. A Python `threading.Lock` lives inside one process and cannot stop another one.

The fix

Move the entire read-decide-write into a **single Lua script executed inside Redis**. Redis runs Lua **single-threaded** — while a script is running, no other command from any client can run. The sequence becomes atomic across every process at once, in one round trip, with no locks.

```
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)
if count >= limit then
    local oldest = redis.call('ZRANGE', key, 0, 0, 'WITHSCORES')
    return {0, count, window - (now - tonumber(oldest[2]))}
end
redis.call('ZADD', key, now, member)
redis.call('PEXPIRE', key, window)
return {1, count + 1, window}
```

How it is verified

`loadtest/burst.py` fires **50 requests simultaneously** at a key with a limit of 10.

- A naive implementation returns 14, or 23, or 31 — a different wrong number every run.
- This returns **exactly 10 allowed and 40 rejected, every run**, across four separate OS processes.

A second race, needing a different fix

The same class of bug appears again at startup. All four uvicorn workers boot simultaneously and all four run `CREATE TABLE` against an empty database; three crash with a duplicate-key error. Redis Lua is no help here, because the contended resource is **Postgres**.

Fixed with a **PostgreSQL advisory lock**: the first worker to ask gets it, the other three block, and by the time they proceed the tables exist.

> Two races, two tools. The fix has to live wherever the shared state lives.

6. The plan cache, and keeping it correct

Every request needs to know which plan an API key is on. Querying Postgres each time would make the rate limiter the slowest thing in the stack, so the plan is cached in Redis for 60 seconds using **cache-aside**: check Redis, fall back to Postgres on a miss, then write the result back.

The staleness problem

Caching introduces a classic failure. A customer **pays to upgrade from Free to Pro**. The `UPDATE` runs in Postgres — but the middleware still reads the cached plan, which says Free. A paying customer keeps getting throttled at 10 requests a minute instead of 100.

How it is handled

1. **Delete the cache key on write**, rather than updating it. Deleting is always correct, because the next read returns to the source of truth. Overwriting the cached value could race between two concurrent plan changes — a second race condition inside the fix.
2. **Order matters: write the database first, delete second.** If you delete first and write second, a reader can slip into the gap, read the old row, and write that stale value straight back into the cache.

3. **The 60-second TTL is a safety net, not the mechanism.** The delete provides correctness; the TTL bounds the damage if an invalidation is ever missed because of a crash.

The race that remains

```
1. Reader misses the cache, reads Postgres     -> gets the OLD plan
2. Writer UPDATEs Postgres, deletes the key    (key already empty)
3. Reader writes the OLD plan into Redis       -> stale until the TTL
```

The TTL bounds this to 60 seconds. The proper fixes are a *delayed double delete* or *versioned cache keys*.

Negative caching

Unknown API keys are cached as missing for 10 seconds. Without this, a flood of bogus keys would hit Postgres on every single request — cache penetration. The shorter TTL means a revoked key stops working quickly.

7. Production details that are easy to get wrong

- **Fail open, not closed.** If Redis is unreachable the request is allowed and the failure logged loudly. Failing closed would mean a Redis hiccup becomes a total outage — the component whose job is protecting availability would be the thing destroying it. Brief unmetered traffic beats downtime. A billing-critical limiter should choose the opposite and say so.
- **Standards-compliant rejection.** HTTP **429** (RFC 9110) with `Retry-After`, plus the `X-RateLimit-*` headers. Well-behaved clients read `Retry-After` and back off automatically; returning 400 or 503 breaks that.
- **Health checks are exempt.** Throttling `/healthz` would make an orchestrator conclude the service is dead and restart it, turning a busy moment into a restart loop.
- **401 vs 403.** No API key at all is **401** ("I do not know who you are"). A key that is unknown or revoked is **403** ("I know who you are and you are not allowed").
- **Synchronous SQLAlchemy runs in a threadpool** so it never blocks the async event loop. Thanks to the cache this path runs only about once per key per TTL.

8. Design patterns used

Pattern	Where	In plain terms
Chain of Responsibility	the middleware	airport security: any station can stop you
Strategy	the two limiters	one `check()`, two algorithms behind it
Factory	`get_limiter()`	picks the strategy from the plan row
Cache-aside	`find_plan()`	check your pocket before going to the ATM

9. API reference

Method	Path	Notes
GET	/healthz	exempt from limiting
GET	/api/ping	rate limited, needs X-API-Key
GET	/api/echo/{v}	rate limited
POST	/admin/keys	create a key on a plan
PATCH	/admin/keys/{key}	change tier, invalidates the cache
GET	/admin/usage	live usage and throttle totals
GET	/docs	auto-generated interactive documentation

On /admin/usage: the active list is a *live snapshot* — counters delete themselves 60 seconds after the last request, so it is empty when idle. throttled_totals is cumulative and never expires.

10. What the test suite demonstrates

Test	Result
50 simultaneous requests, limit 10	exactly 10 allowed, across 4 processes
Identical burst, different plans	free gives 10, pro gives 100
Live plan change mid-run	10 of 30, then 30 of 30 after upgrading
Rejection headers	Retry-After and all X-RateLimit-* present
Unknown key / missing key	403 / 401

The live plan change is also the **regression test for cache invalidation**: if the cache were not cleared on the write, the second burst would still be capped at 10.

Running it: install Docker Desktop, then double-click run.bat. See RUNBOOK.md for the manual steps and troubleshooting.